Bilkent University

Department of Computer Engineering

Senior Design Project

Low Level Design Report

Bil-Events : A mobile app to organize events in Bilkent University

Team Members

Osman Can Yıldız -21302616

Kasım Sarp Ataş - 21502006

Can Savcı - 21300803

Supervisor: Halil Altay Güvenir

Jury Members: Fazlı Can and Özgür Ulusoy

Innovation Expert: Barış Misman

Low Level Design Report
Oct. 5, 2020

This report is submitted to the Department of Computer Engineering of Bilkent University in partial fulfillment of the requirements of the Senior Design Project course CS491/492

1. Introduction

According to YÖK's website, there are currently over 13.000 students studying in Bilkent [1]. Most of these students are members of some student clubs. These clubs host many different events for Bilkenters. However, there are several problems with the event notification system. First of all, since there is no unified notification system that people can access the information about events from various places. There are three main ways to access event information. One of them is from webmail notifications (BAIS), another one is from students clubs' posters and the third one is from SRS' GE 250/251 notifications. However, all of these three have their own flaws. To clarify what these flaws are actually, we should examine the event notification systems. The first one, the BAIS mail system, which sends over ten mail per day about not only events but also all notifications about Bilkent University and the seminars. It is hard to keep track of events and remember their dates. In addition to this, the BAIS mail system sends the emails unaesthetically, it causes resentment even for the students who want to invest their valuable time by attending the events. The second one, using paper posters is outdated, also a huge waste of paper and they are not well-distributed. They are so randomly distributed that it causes distraction among the students who want to find the right event for themselves. For instance, you can't find specific events posters in every building as they are not hung everywhere or they are overlapped when they are hung. For the students in the East Campus, this issue cannot even be mentioned because they cannot reach information about the events, because the students don't have any idea how to find the locations of the events and the events are generally held in the Main Campus, the distance between East Campus and the location of the events and lack of knowledge on how to find the location give resentment to the students who are in East Campus. BAIS e-mails and posters are not descriptive enough to handle that problem we mentioned. The last point we want to mention is that the update on SRS' event notification system does not even show the events that don't give GE 250/251 points. As a group of students who are in this system, we observed and suffered from these problems and as a solution we offer an app which will be accessed by everyone.

1.1. Object Design Trade-offs

1.1.1 Security vs. Cost

Bil-Events collects several data from users, which is composed of private information of different students. Thus, it is crucial for the system to ensure security and keep the information of the users secure. For security, we rely on encrypted databases. Relatedly, security introduces monetary, time, and labour cost.

1.1.2 Space vs. Speed

We will implement the application using React Native on the front-end part and Python on the back-end part. Since, Python has a Compiler and an Interpreter, the application will be slower than the implementations using C-family programming languages. In addition to this, the large amount of data introduces difficulties in saving and fetching the data from the servers. However, all connections and data exchanges with the server will be handled in caching, database optimization and template performance. Often it is expensive (that is, resource-hungry and slow) to compute a value, so there can be huge benefit in saving the value to a quickly accessible cache, ready for the next time it's required. It's a sufficiently significant and powerful technique that Django includes a comprehensive caching framework, as well as other smaller pieces of caching functionality. Django's caching framework offers very significant opportunities for performance gains, by saving dynamic content so that it doesn't need to be calculated for each request. For convenience, Django offers different levels of cache granularity: you can cache the output of specific views, or only the pieces that are difficult to produce, or even an entire site. Implementing caching should not be regarded as an alternative to improving code that's performing poorly because it has been written badly. It's one of the final steps towards producing well-performing code, not a shortcut. Alternatively, using cached sessions may be a way to increase performance by eliminating the need to load session data from a slower storage source like the database and instead storing frequently used session data in memory. Django's database layer provides various ways to help developers get the best performance from their databases. The database optimization documentation gathers together links to the relevant documentation and adds various tips that outline the steps to take when attempting to optimize your database usage. Using various ways mentioned above, we will be able to keep the system fast and highly responsive while managing lots of data at the same time.

1.2. Engineering standards (e.g., UML and IEEE)

In all documentations, UML design principles are used in the description of class interfaces, diagrams, scenarios and use cases, subsystem compositions, and hardware-software components depiction. The goal of UML is to provide a standard notation that can be used by all object-oriented methods and to select and integrate the best elements of precursor notations. UML has been designed for a broad range of applications. Hence, it provides constructs for a broad range of systems and activities (e.g., distributed systems, analysis, system design and deployment).

1.3. Interface Documentation Guidelines

All of the classes in the view module are documented by ES6 standard since we are implementing the front-end of the application using React Native and all of the classes in the model and controller modules are documented by PEP8 standard since we are implementing the back-end of the application using Python. All of the classes, variables and methods that are going to be used in the system will use Camel Case convention in terms of naming. Every class will follow the same design pattern where the class name comes first, then the attributes are presented and lastly the methods are listed.

1.4. Definitions, acronyms, and abbreviations

UI: User Interface

API: Application Programming Interface

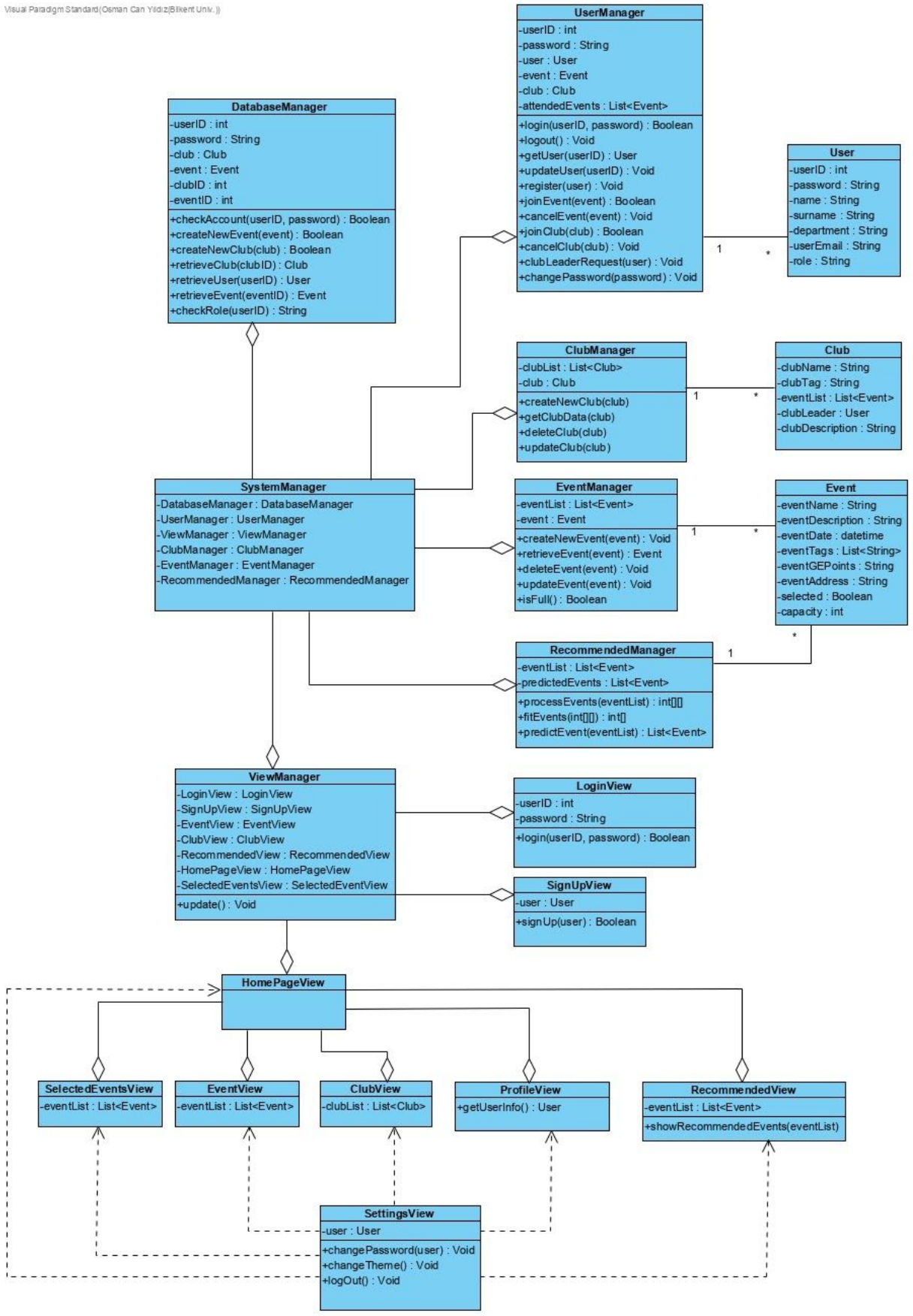
HTTP: Hypertext Transfer Protocol

TCP: Transmission Control Protocol

Client: The part of the system that the user interacts with.

Server: The part of the system that responds to client's requests. It is responsible for data management, API interactions and logical operations.

2. Packages

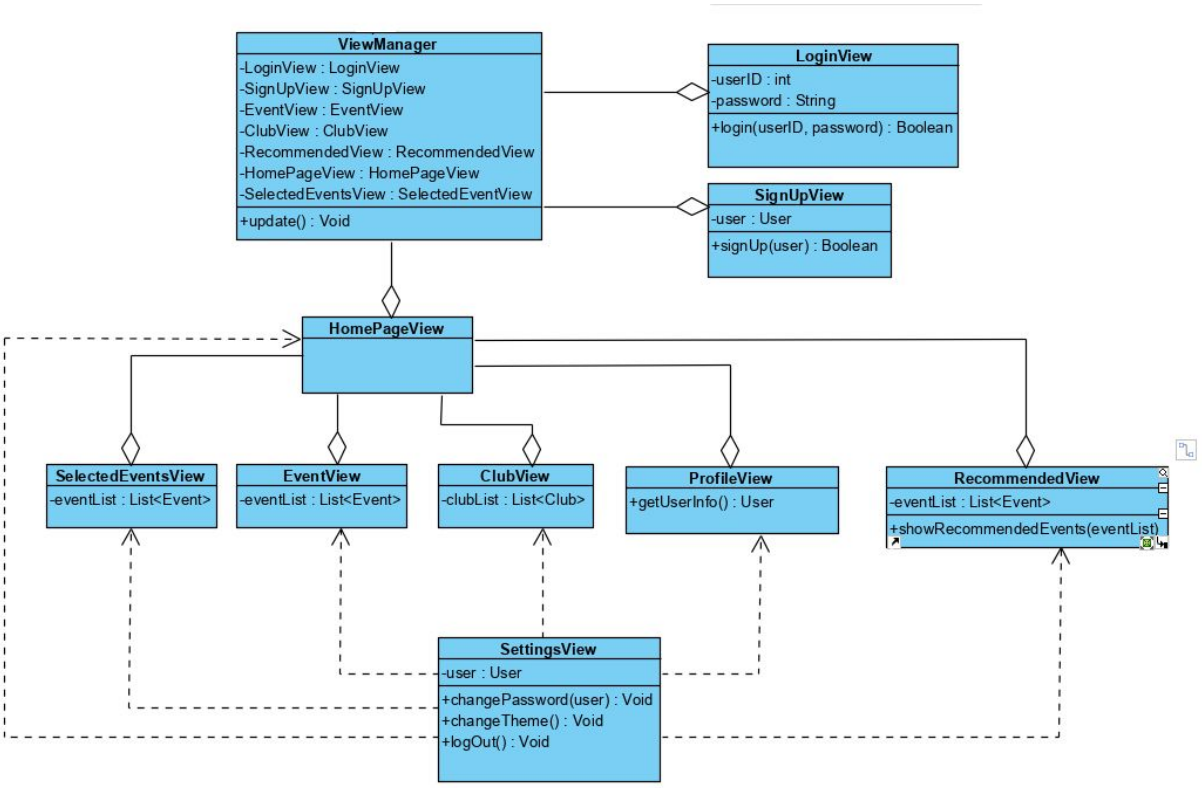

Visual Paradigm Standard(Osman Can Yildiz(Bilkent Univ.))
UserManager
-userID : int
-password : String
-user : User
-event : Event
-club : Club
-attendedEvents : List<Event>
+login(userID, password) : Boolean
+logout() : Void
+getUser(userID) : User
+updateUser(userID) : Void
+register(user) : Void
+joinEvent(event) : Boolean
+cancelEvent(event) : Void
+joinClub(club) : Boolean
+cancelClub(club) : Void
+clubLeaderRequest(user) : Void
+changePassword(password) : Void
DatabaseManager
-userID : int
-password : String
-club : Club
-event : Event
-clubID : int
-eventID : int
+checkAccount(userID, password) : Boolean
+createNewEvent(event) : Boolean
+createNewClub(club) : Boolean
+retrieveClub(clubID) : Club
+retrieveUser(userID) : User
+retrieveEvent(eventID) : Event
+checkRole(userID) : String
User
-userID : int
-password : String
-name : String
-surname : String
-department : String
-userEmail : String
-role : String
1
*
ClubManager
-clubList : List<Club>
-club : Club
+createNewClub(club)
+getClubData(club)
+deleteClub(club)
+updateClub(club)
Club
-clubName : String
-clubTag : String
-eventList : List<Event>
-clubLeader : User
-clubDescription : String
SystemManager
-DatabaseManager : DatabaseManager
-UserManager : UserManager
-ViewManager : ViewManager
-ClubManager : ClubManager
-EventManager : EventManager
-RecommendedManager : RecommendedManager
EventManager
-eventList : List<Event>
-event : Event
+createNewEvent(event) : Void
+retrieveEvent(event) : Event
+deleteEvent(event) : Void
+updateEvent(event) : Void
+isFull() : Boolean
Event
-eventName : String
-eventDescription : String
-eventDate : datetime
-eventTags : List<String>
-eventGEPoints : String
-eventAddress : String
-selected : Boolean
-capacity : int
RecommendedManager
-eventList : List<Event>
-predictedEvents : List<Event>
+processEvents(eventList) : int[][]
+fitEvents(int[][]) : int[]
+predictEvent(eventList) : List<Event>
ViewManager
-LoginView : LoginView
-SignUpView : SignUpView
-EventView : EventView
-ClubView : ClubView
-RecommendedView : RecommendedView
-HomePageView : HomePageView
-SelectedEventsView : SelectedEventView
+update() : Void
LoginView
-userID : int
-password : String
+login(userID, password) : Boolean
SignUpView
-user : User
+signUp(user) : Boolean
HomePageView
SelectedEventsView
-eventList : List<Event>
EventView
-eventList : List<Event>
ClubView
-clubList : List<Club>
ProfileView
+getUserInfo() : User
RecommendedView
-eventList : List<Event>
+showRecommendedEvents(eventList)
SettingsView
-user : User
+changePassword(user) : Void
+changeTheme() : Void
+logOut() : Void

Current designs are not the exact solutions to the problem, because they have a hard time presenting the information to the students. The current systems are not user friendly, it is not organized and it is presented with the other emails or irrelevant information. Our application is portable and presentable to get the information of the events efficiently. Combining the portability, the specialization of event representations and the recommendation system will make our platform preferable. Below we will explain our two subsystem design in a more detailed way.

2.1. Client

Our client subsystem is composed of two tiers of labels: User Interface Tier and Control Tier. The UI Tier interacts with the end user and controls the interactions in order to respond accurately to the end users call and to communicate with the server, it has the Control Tier. Our application will be an Android application, we have designed the client subsystem accordingly. Client Subsystem will be responsible for all of the user operations, sending and receiving data, notifications, displaying information about the events etc. End users can make different requests and our Client Subsystem can handle accordingly.

2.1.1. Presentation Package



Inside the Presentation Package, the application handles all of the user input operations. The tier consists of the following classes.

LoginView: The class handles the request of viewing the login page of the application

SignupView: This class handles the Sign-Up operations and UI for users that are not yet registered to our system.

ProfileView: The class handles the request of viewing the personal profile of the user of the application.

HomePageView: This class handles all Home Page related operations and the UI.

SelectedEventView: The class handles the request of viewing the selected events in the application.

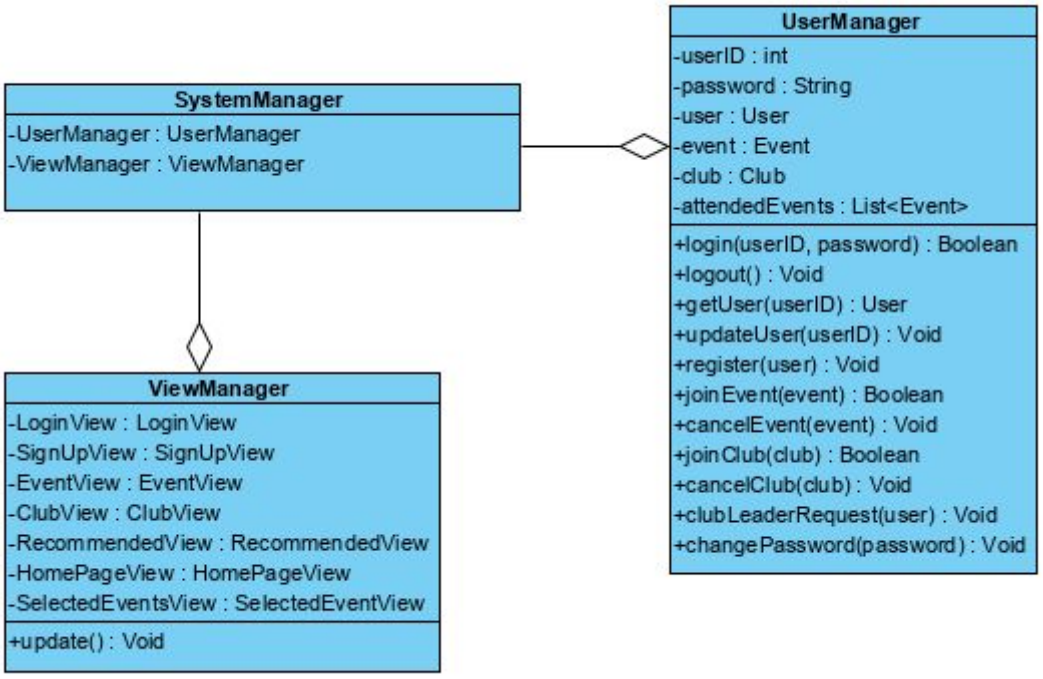
EventView: The class handles the request of viewing the events in the application.

ClubView: The class handles the request of viewing the list of all available clubs in the application.

RecommendedView: The class handles the request of viewing the recommended events in the application.

SettingsView: The interface handles the request of viewing the list of settings on the right top corner of the application.

2.1.2. Control Tier



The Control Tier handles all of the saving and receiving data operations and it is directly connected to the server. The tier consists of the following classes.

ViewManager: Class that controls all views of the system and navigations the change from one to another.

UserManager: Class that controls all of the interactions between the user and the client.

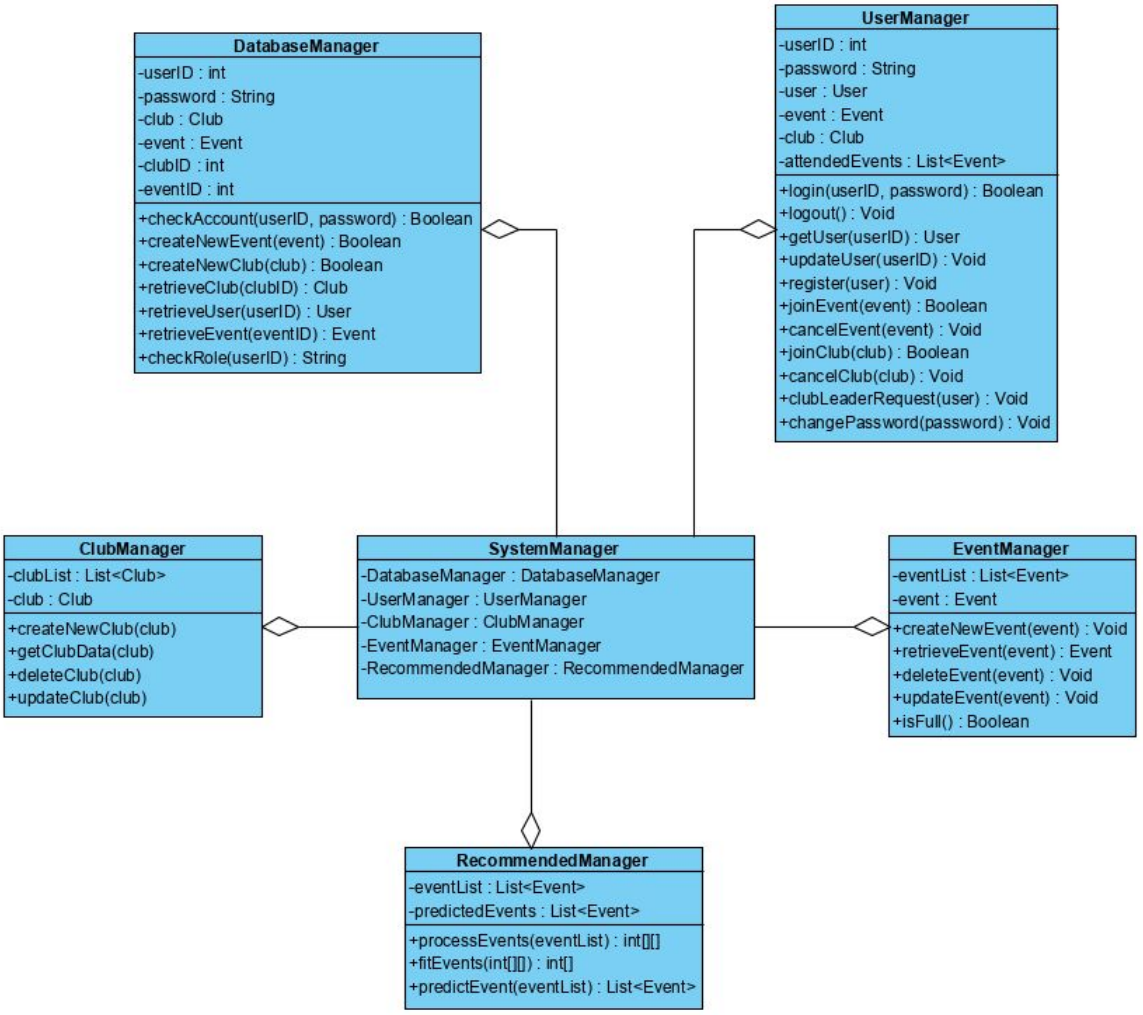
SystemManager: The class creates the connection between the client and the server.

2.2. Server Subsystem

The Server subsystem is composed of two tiers: Logic Tier and Data Tier. Logic Tier is responsible for getting all the requests, processing them and filtering all the data as well. Data Tier is responsible for storing the data. The information of new users, new events or new clubs are sent by the client and after the processing part the data is sent to the server. Because, some of the requests may contain irrelevant information that will be processed and checked by this tier. For instance, the students from other universities can access the application and the students must be checked whether the students are from Bilkent University or not. Another example could be the addition of non-existent clubs to the application or forgetting the deletion of inactive clubs; therefore, such small details like the examples above need to be checked.

After the addition of the information of any event, any club or any student is successfully done, the students can access the events in detail, they can see the upcoming events, the selected events and the recommended events as well . For accessing the event, the server part is responsible since the client is requesting. Server side by using the secure database implementation can detect phishing attempts of any violent user. In this way, the server subsystem controls and secures the data and executes all the operations with them.

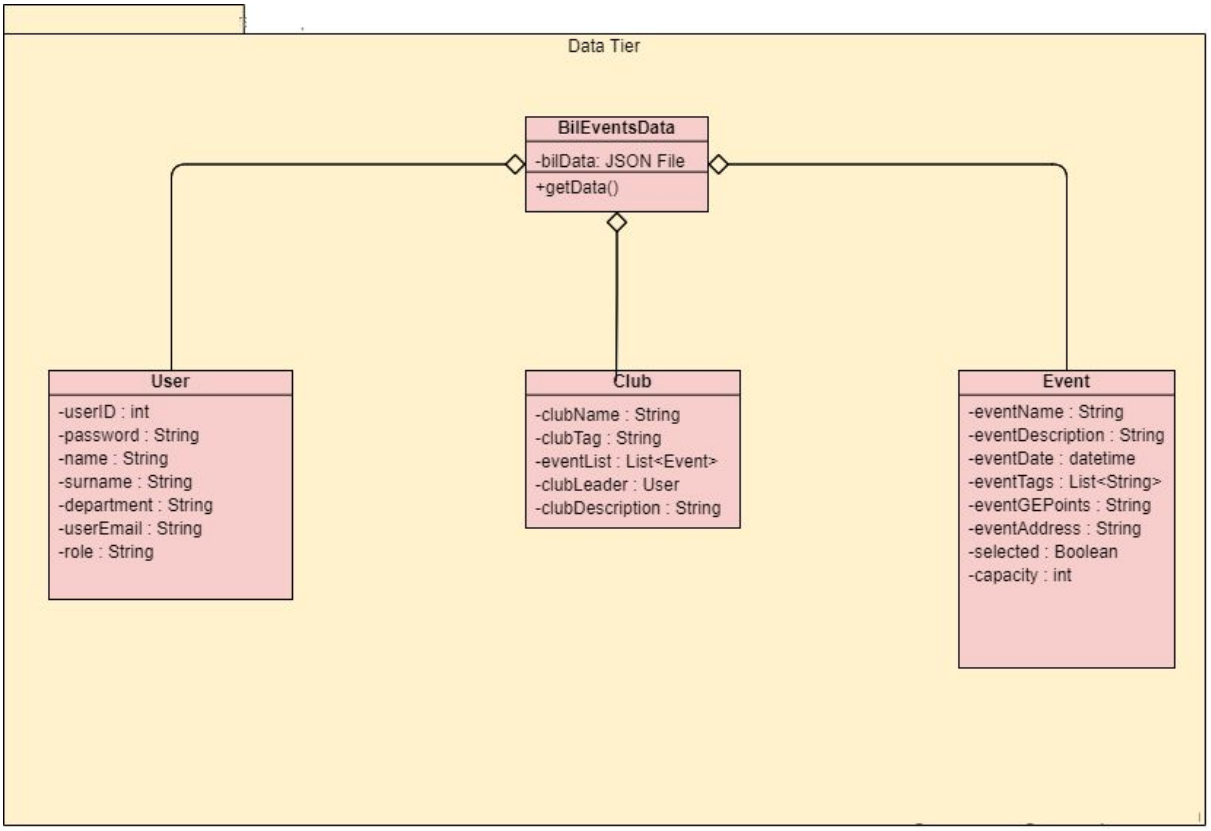
2.2.1. Logic Tier



Logic Tier is responsible for processing and filtering all of data before saving them in the database or making a prediction to generate data using existing data segments. The classes of Logic Tier are explained in below.

EventManager: The class checks the validity of the events of the application.
UserManager: The class checks the validity of the users (e.g. whether the user is a member in Bilkent University) in the application.
ClubManager: The class checks the validity of the clubs in the Bilkent University, in the application.
RecommendedManager: The class makes the prediction of the events that the user may possibly prefer to attend by using the previous attended events of the user.
SystemManager: The class creates the connection between all controllers.
DatabaseManager: The class executes the CRUD changes of Data Tier in the database

2.2.2. Data Tier



Data Tier stores all of the raw data by organizing them and handles all of the queries when the data is needed to be accessed. The tier consists of the following classes.

Users: The class stores the users' information in the application.
Events: The class stores the events' information in the application.
Clubs: The class stores the clubs' information in the application.

3. Class Interfaces

3.1. Client

3.1.1. Presentation Tier

Presentation Tier is responsible for all of the user interface interactions and it uses Control Tier in order to communicate with the server.

Class LoginView	
Attributes	
-userID : int -password: string	
Methods	
+login(userID,password): boolean	This method redirects the user to the login page.

Class SignUpView	
Attributes	
-user: user	
Methods	
+signUp(user): Boolean	This method redirects the user to the sign up page.

Class ProfileView	
Methods	
+getUserInfo(): user	This method gets the user information for the profile page.

Class HomePageView

Class SelectedEventsView
Attributes
-eventList: List<Event>

Class EventView
Attributes
-eventList: List<Event>

Class ClubView
Attributes
-clubList: List<Club>

Class RecommendedView	
Attributes	
-eventList: List<Event>	
Methods	
+showRecommendedEvents(eventList)	This method redirects the user to the recommended events page.

Class SettingView	
Attributes	
-user: user	
Methods	
+goToUserProfile(): void	This method redirects the user to the profile page.
+changePassword(user): void	This method redirects the user to the change password page.
+changeTheme(): void	This method changes the theme color.
+logOut(): void	This method redirects the user to the login page as loggedOut.

3.1.2. Controller Tier

Class SystemManager
Attributes
-DatabaseManager : DatabaseManager -UserManager : UserManager -ViewManager: ViewManger -ClubManager : ClubManager -EventManager : EventManager

RecommendedManager: RecommendedManager

Class ViewManager	
Attributes	
-LoginView : LoginView -SignUpView : SignUpView -EventView : EventView -ClubView : ClubView -RecommendedView : RecommendedView -HomePageView : HomePageView -SelectedEventView : SelectedEventView	
Methods	
update() : void	This method updates the views.

Class EventManager	
Attributes	
-eventList : List<Event> -event Event	
Methods	
updateEvent(event) : void	This method updates an existing event in the database.
deleteEvent(event) : void	This method deletes an event from the database.
createNewEvent(event) : void	This method adds a new event to the database.
retrieveEvent(event) : Event	This method gets an event from the database.

Class ClubManager
Attributes
-clubList : List<Club> -club: Club
Methods

updateClub(club) : void	This method updates an existing club in the database.
deleteClub(club) : void	This method deletes a club from the database.
createNewClub(club) : void	This method adds a new club to the database.
retrieveEvent(event) : Event	This method gets a club from the database.

Class UserManager	
Attributes	
-userID : int -password : String -user: user -event: Event -club :Club -attendedEvents : List<Event>	
Methods	
+login(userID,password) : Boolean	The method which gives the authentication to the user if the ID of user and password matches.
+logout(): void	The method which helps user to log out of the application.
+getUser(userID): User	This method gets the user from the database.
+updateUser(userID): void	This method updates an existing user in the database.
+register(user) : void	This method stores the newly registered user to the database.
+joinEvent(event): Boolean	The method which assigns the user to the particular event.
+cancelEvent(event) : void	The method which revokes the user from the particular event.
+joinClub(club): Boolean	The method which assigns the user to the particular club.
+cancelClub(club): void	The method which revokes the user from the particular club.
+clubLeaderRequest(user) : void	The method which assigns the club leader

	role to the user with the given permission.
+changePassword(password): void	The method which changes the password of the user with the given authorization.

Class DatabaseManager	
Attributes	
-userID : int -password : String -event: Event -club :Club -clubID: int -eventID: int	
Methods	
+checkAccount(userID,password): Boolean	The method which is responsible for authorization of the user.
+createNewEvent(event): Boolean	The method which creates a new event given the event parameters.
+createNewClub(club): Boolean	The method which creates a new club given the club parameters.
+retrieveClub(clubID): Club	The method which retrieves the particular club from an id of a club.
+retrieveUser(userID): User	The method which retrieves the particular user from an id of a user.
+retrieveEvent(eventID): Event	The method which retrieves the particular event from an id of an event.
+checkRole(userID): String	The method which checks the validity of the role of the user.

3.2. Server

3.2.1. Logic Tier

Class RecommendedManager
Attributes
-eventList : List<Event> -predictedEvents : List<Event>

Methods	
processEvents(eventList): int [ ] [ ]	The method which takes the list of events and creates a 2D matrix the count of attended events with respect to the row which consists of the users and the column which consists of the event tags.
fitEvents(int [ ] [ ]): int [ ]	The method which helps to fit the data of previous events.
predictedEvent(eventList) : List<Event>	The method which predicts and extracts the preferred events given new events.

3.2.2. Data Tier

Class BilEventsData	
Attributes	
-bilData : JSON File	
Methods	
getData() : JSON	The method which obtains and represents the data in a JSON format.

Class User
Attributes
-userID : int -password : String -name : String -surname : String -department : String -userEmail : String -role : String

Class Club

Attributes
-clubName : String -clubTag : String -eventList : List <Event> -clubLeader : User -clubDescription : String

Class Event
Attributes
-eventName : String -eventDescription : String -eventDate : String -eventTags : List<String> -eventGEPoints : String -eventAddress : String -selected : Boolean -capacity : int

4. References

[1]"Performance and optimization | Django documentation | Django", *Docs.djangoproject.com*, 2020. [Online]. Available: https://docs.djangoproject.com/en/3.1/topics/performance/. [Accessed: 21- Sep- 2020].

[2]"Client-Side vs. Server-Side Code: What's the Difference?", *Segue Technologies*, 2020. [Online]. Available: https://www.seguetech.com/client-server-side-code/. [Accessed: 21- Sep- 2020].

[3]"Introduction to recommender systems", *Medium*, 2020. [Online]. Available: https://towardsdatascience.com/introduction-to-recommender-systems-6c66cf15ada. [Accessed: 21- Sep- 2020].